Lab Manual: Deploying a Streamlit App on Azure Virtual Machine with Docker

Objective

By the end of this lab, you will:

- Package your Streamlit app into a Docker container.
- Launch and configure an **Azure Virtual Machine** (Ubuntu).
- Deploy and run the app inside Docker.
- Access the app publicly using the VM's public IP.

Prerequisites

- A working Streamlit app (`app.py`).
- Python installed locally.
- Docker installed locally (optional, for pre-testing).
- An **Azure account** with VM creation privileges.
- Basic familiarity with SSH and terminal commands.

Step 1: Prepare Your Streamlit App

1. Make sure your main entry file is: `app.py`
2. Export dependencies into `requirements.txt`: `pip freeze > requirements.txt`

Captures all Python libraries your app needs.

Step 2: Create a Dockerfile

In your project folder, create a file named `Dockerfile`:

```
# Base Python image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy app code
COPY . .

# Expose Streamlit default port
EXPOSE 8501

# Run Streamlit app
CMD ["streamlit", "run", "app.py", "--server.port=8501", "--
server.address=0.0.0.0"]
```

Explanation:

- `python:3.9-slim` → lightweight base image.
- `EXPOSE 8501` → opens the default Streamlit port.
- `CMD` → runs Streamlit on `0.0.0.0:8501`, accessible externally.

Step 3: Launch an Azure Virtual Machine

1. Go to **Azure Portal → Virtual Machines → Create VM**.
2. Choose:
    a. **Image**: Ubuntu Server 20.04 LTS (recommended).
    b. **Size**: Standard_B1s (or higher).
    c. **Authentication**: SSH key.

3. Configure **Networking → Inbound port rules**:
    a. Allow **TCP traffic on port 8501** (for Streamlit).
    b. Allow **port 22** (for SSH access).
4. Review and **Create** the VM.
5. Copy the **public IP** of your VM (used later).

Step 4: Connect to the Azure VM

On your local machine:

```
ssh -i my-key.pem azureuser@<AVM_PUBLIC_IP>
```

(Replace `azureuser` with your VM username and `<AVM_PUBLIC_IP>` with your VM's public IP.)

Step 5: Install Docker on the VM

On the Azure VM:

```
# Update system
sudo apt update -y

# Install Docker
sudo apt install -y docker.io

# Start and enable Docker
sudo systemctl start docker
sudo systemctl enable docker
```

Docker is now running on your VM.

Step 6: Transfer App Files to the VM

From your local machine:

```
scp -i my-key.pem -r ./my-streamlit-app
azureuser@<AVM_PUBLIC_IP>:/home/azureuser/
```

Your app is now copied into `/home/azureuser/my-streamlit-app` on the VM.

Step 7: Build and Run the Docker Container

On the Azure VM:

```
# Go to app folder
cd my-streamlit-app

# Build Docker image
sudo docker build -t streamlit-avm .

# Run container in detached mode
sudo docker run -d -p 8501:8501 streamlit-avm
```

Your Streamlit app is running inside Docker on port **8501**.

Step 8: Access the App in Browser

Open in your browser: `http://<AVM_PUBLIC_IP>:8501`. You should now see your Streamlit app live on Azure.

Verification Checklist

- `app.py`, `requirements.txt`, and `Dockerfile` created.
- Azure VM launched with ports **22** and **8501** open.
- Docker installed and running on VM.
- App copied to VM successfully.
- Docker image built and container running.
- App accessible via `http://<AVM_PUBLIC_IP>:8501`.

Summary

In this lab, you:

- Packaged your Streamlit app in a **Docker container**.
- Launched an **Azure Virtual Machine**.
- Installed Docker and deployed the app.
- Accessed it publicly using the VM's IP.

This setup provides **flexibility and scalability,** you can easily adjust VM resources or extend your container with extra services.